

Architekturdokument

DecisionLab – Entscheidungsunterstützung für multikriterielle Entscheidungen

1 Technologieauswahl

Für die Implementierung von DecisionLab wird die Programmiersprache Python verwendet. Python unterstützt die objektorientierte Softwareentwicklung und ermöglicht eine übersichtliche Strukturierung der für DecisionLab benötigten Geschäfts-, Persistenz- und Benutzeroberflächenlogik.

Für die grafische Benutzeroberfläche wird Tkinter eingesetzt. Tkinter ermöglicht die Entwicklung einer lokalen Desktopanwendung und stellt die für DecisionLab benötigten grundlegenden GUI-Komponenten wie Fenster, Schaltflächen, Eingabefelder, Auswahl Elemente und tabellarische Darstellungen bereit. Da DecisionLab als lokale Einzelbenutzeranwendung konzipiert ist und keine browserbasierte Oberfläche benötigt, ist kein zusätzliches Webframework erforderlich.

Zur persistenten Speicherung der Anwendungsdaten wird SQLite verwendet. Die Datenbank wird lokal gespeichert und benötigt keinen separaten Datenbankserver. Dies entspricht der Anforderung, Entscheidungen einschließlich Alternativen, Kriterien, Gewichtungen und Bewertungen lokal zu speichern und nach einem Neustart der Anwendung wiederherzustellen.

Für die Versionsverwaltung und Bereitstellung des Quellcodes wird Git in Verbindung mit einem GitHub-Repository eingesetzt. Dadurch können Entwicklungsstände nachvollziehbar versioniert und die im Projekt entstehenden Quell- und Dokumentationsdateien zentral bereitgestellt werden.

Die gewählte Technologiekombination ermöglicht damit eine schlanke Umsetzung der spezifizierten Desktopanwendung ohne zusätzliche Server- oder Cloudinfrastruktur.

2 Softwarearchitektur

2.1 Architekturkonzept

DecisionLab wird nach einer einfachen Schichtenarchitektur strukturiert. Ziel ist es, die Benutzeroberfläche, die fachliche Anwendungslogik und die persistente Datenhaltung voneinander zu trennen. Dadurch sollen Änderungen an einzelnen Teilen der Anwendung möglichst geringe Auswirkungen auf andere Bereiche haben und zentrale fachliche Funktionen unabhängig von der grafischen Benutzeroberfläche testbar bleiben.

Die Benutzeroberflächenschicht übernimmt die Darstellung der Anwendungsdaten sowie die Verarbeitung von Benutzereingaben. Sie enthält keine eigenständige Berechnungslogik für Gesamtscores oder Sensitivitätsanalysen.

Die Geschäftslogikschicht enthält die fachlichen Operationen der Anwendung. Dazu gehören insbesondere die Validierung fachlicher Voraussetzungen, die Berechnung der gewichteten Gesamtscores, die Bildung der Rangfolge sowie die Berechnung temporärer Gewichtungen und Ergebnisse für die Sensitivitätsanalyse.

Die Persistenzschicht übernimmt das Speichern, Laden, Ändern und Löschen der Anwendungsdaten in einer lokalen SQLite-Datenbank.

Ergänzend werden die zentralen fachlichen Daten durch Modellklassen repräsentiert. Hierzu gehören die bereits in der Spezifikation definierten Geschäftsobjekte Decision, Alternative, Criterion und Rating.

2.2 Aufbau der Architektur

Die Anwendung wird in folgende Bereiche gegliedert:

Benutzeroberfläche (UI): Darstellung der verschiedenen Ansichten und Dialoge sowie Entgegennahme der Benutzereingaben.

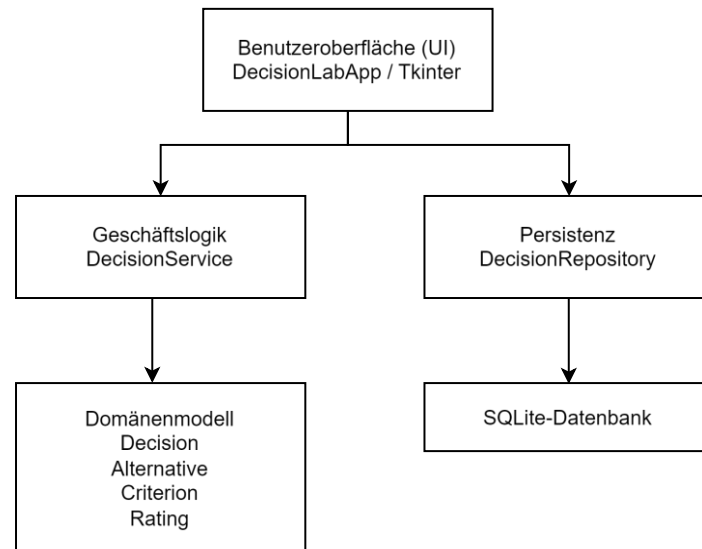
Geschäftslogik (Services): Verarbeitung und Validierung der Eingaben sowie Durchführung der Berechnungen und Sensitivitätsanalysen.

Domänenmodell (Models): Objektorientierte Repräsentation von Entscheidungen, Alternativen, Kriterien und Bewertungen.

Persistenz (Repository): Zugriff auf die lokale SQLite-Datenbank und dauerhafte Speicherung der Anwendungsdaten.

Die Benutzeroberfläche verwendet die Geschäftslogik zur Durchführung fachlicher Operationen und die Persistenzschicht zum Speichern und Laden der Anwendungsdaten. Die Geschäftslogik arbeitet mit den Modellobjekten und enthält keine direkten Datenbankzugriffe. Der Zugriff auf die SQLite-Datenbank erfolgt ausschließlich über die Persistenzschicht.

Abbildung 1: Architekturübersicht von DecisionLab



Quelle: Eigene Darstellung

3 Komponenten und Klassen

3.1 Modellklassen

Die Modellklassen bilden die fachlichen Geschäftsobjekte der Anwendung ab. Ihre grundlegende Struktur orientiert sich am bereits in der Spezifikation definierten fachlichen Datenmodell. Decision repräsentiert eine Entscheidung, Alternative eine zugehörige Auswahlmöglichkeit, Criterion ein Bewertungskriterium einschließlich seiner Gewichtung und Rating die Bewertung einer Alternative bezüglich eines Kriteriums.

Die Modellklassen enthalten primär die Daten der jeweiligen Geschäftsobjekte. Umfangreiche fachliche Berechnungen werden nicht innerhalb der Modellklassen durchgeführt, sondern zentral durch den DecisionService.

3.2 DecisionService

Die Klasse DecisionService bündelt die zentrale fachliche Logik der Anwendung. Sie prüft, ob die Voraussetzungen für eine Auswertung erfüllt sind, berechnet die gewichteten Gesamtscores und erzeugt die daraus resultierende Rangfolge. Darüber hinaus stellt sie die Berechnungslogik für die Sensitivitätsanalyse bereit.

Die Berechnungen werden dadurch von der Benutzeroberfläche und der Datenhaltung getrennt. Dies ermöglicht insbesondere, die mathematische Berechnungslogik unabhängig von der grafischen Oberfläche anhand definierter Testfälle zu überprüfen.

3.3 DecisionRepository

Die Klasse DecisionRepository kapselt den Zugriff auf die lokale SQLite-Datenbank. Sie stellt Operationen zum Speichern, Laden und Löschen von Entscheidungen sowie der zugehörigen Alternativen, Kriterien und Bewertungen bereit.

Durch die Kapselung des Datenbankzugriffs müssen weder die Benutzeroberfläche noch die fachliche Berechnungslogik direkte SQL-Anweisungen ausführen. Änderungen an der Persistenz können dadurch weitgehend unabhängig von der übrigen Anwendung vorgenommen werden.

3.4 DecisionLabApp

Die Klasse DecisionLabApp bildet den zentralen Einstiegspunkt der grafischen Benutzeroberfläche. Sie initialisiert das Hauptfenster und koordiniert die Navigation zwischen Startansicht, Entscheidungsbearbeitung, Bewertungsansicht, Ergebnisansicht und Sensitivitätsanalyse.

Benutzereingaben werden von der Oberfläche entgegengenommen und an die zuständigen Komponenten weitergegeben. Für fachliche Berechnungen verwendet die Benutzeroberfläche den DecisionService. Persistente Operationen werden über das DecisionRepository durchgeführt.

3.5 Klassenstruktur und Verantwortlichkeiten

Die nachfolgend beschriebenen Klassen bilden die wesentlichen Bestandteile der geplanten Implementierung. Die Modellklassen repräsentieren die fachlichen Daten, während der DecisionService die zentrale Geschäftslogik und das DecisionRepository die Persistenz kapseln. Die Klasse DecisionLabApp koordiniert die grafische Benutzeroberfläche. Die Klassenstruktur orientiert sich damit an der zuvor beschriebenen Trennung von Benutzeroberfläche, Geschäftslogik, Domänenmodell und Persistenz.

Die Klasse Decision repräsentiert eine Entscheidung und enthält deren eindeutigen Identifikator, Namen und optionale Beschreibung. Die Klasse Alternative repräsentiert eine Auswahlmöglichkeit und enthält neben Identifikator und Namen die Zuordnung zur zugehörigen Entscheidung. Die Klasse Criterion bildet ein Bewertungskriterium mit Identifikator, Namen, Gewichtung und Zuordnung zur Entscheidung ab. Die Klasse Rating repräsentiert die Bewertung einer Alternative hinsichtlich eines bestimmten Kriteriums und enthält den entsprechenden Bewertungswert.

Der DecisionService enthält die zentrale fachliche Logik der Anwendung. Die Methode `validate_for_evaluation()` prüft, ob die Voraussetzungen für eine Auswertung erfüllt sind. Hierzu gehören insbesondere die erforderliche Anzahl von Alternativen und Kriterien, gültige Kriteriengewichtungen, eine Gewichtungssumme von 100 % sowie vollständige und gültige Bewertungen. Die Methode `calculate_scores()` berechnet die gewichteten Gesamtscores der Alternativen. `create_ranking()` erzeugt auf Grundlage der berechneten Gesamtscores die

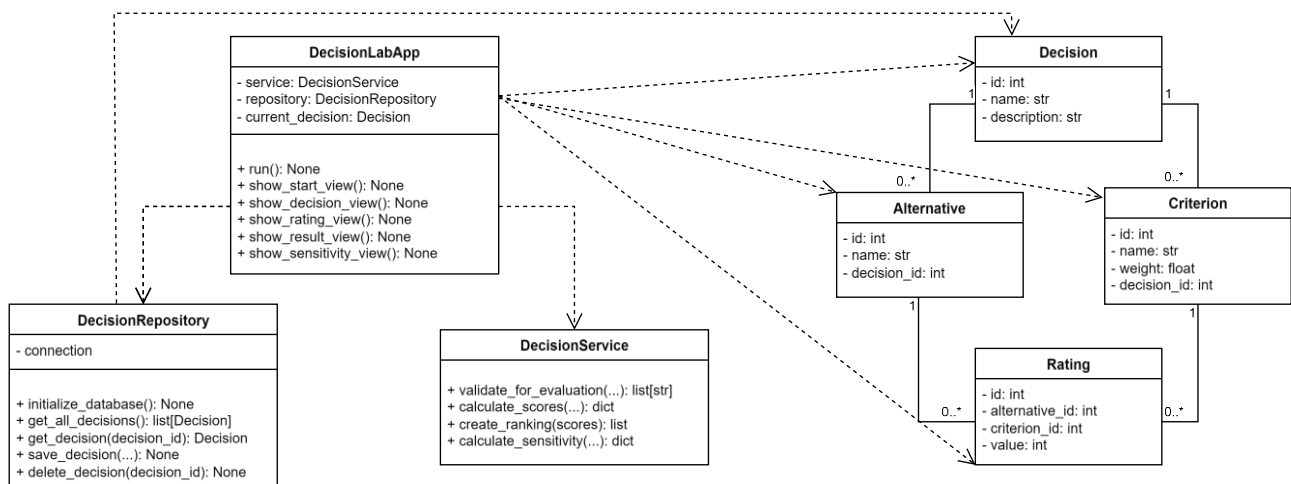
Rangfolge und berücksichtigt dabei auch Alternativen mit identischem Gesamtscore. Die Methode `calculate_sensitivity()` führt die Sensitivitätsanalyse mit einer temporär veränderten Kriteriengewichtung durch und liefert die daraus resultierenden Gewichtungen, Gesamtscores und Rangfolge. Die ursprünglich gespeicherten Gewichtungen werden dabei nicht verändert.

Das `DecisionRepository` kapselt den Zugriff auf die lokale SQLite-Datenbank. `initialize_database()` stellt die benötigte Datenbankstruktur bereit. `get_all_decisions()` lädt die für die Startansicht benötigte Übersicht vorhandener Entscheidungen. `get_decision()` lädt eine ausgewählte Entscheidung und die zugehörigen Daten. `save_decision()` speichert eine Entscheidung einschließlich ihrer Alternativen, Kriterien und Bewertungen. `delete_decision()` entfernt eine Entscheidung und die ihr zugeordneten Daten.

Die Klasse `DecisionLabApp` bildet den zentralen Einstiegspunkt der grafischen Benutzeroberfläche. Sie verwaltet den aktuellen Anwendungszustand und koordiniert die Navigation zwischen den einzelnen Ansichten. Hierzu gehören insbesondere die Startansicht, die Entscheidungsansicht, die Bewertungsansicht, die Ergebnisansicht und die Sensitivitätsanalyse. Für fachliche Berechnungen verwendet `DecisionLabApp` den `DecisionService`; Datenbankoperationen werden über das `DecisionRepository` ausgeführt.

Die Beziehungen, Attribute und wesentlichen Methoden dieser Klassen werden im nachfolgenden UML-Klassendiagramm dargestellt.

Abbildung 2: UML-Klassendiagramm der Softwarearchitektur von DecisionLab



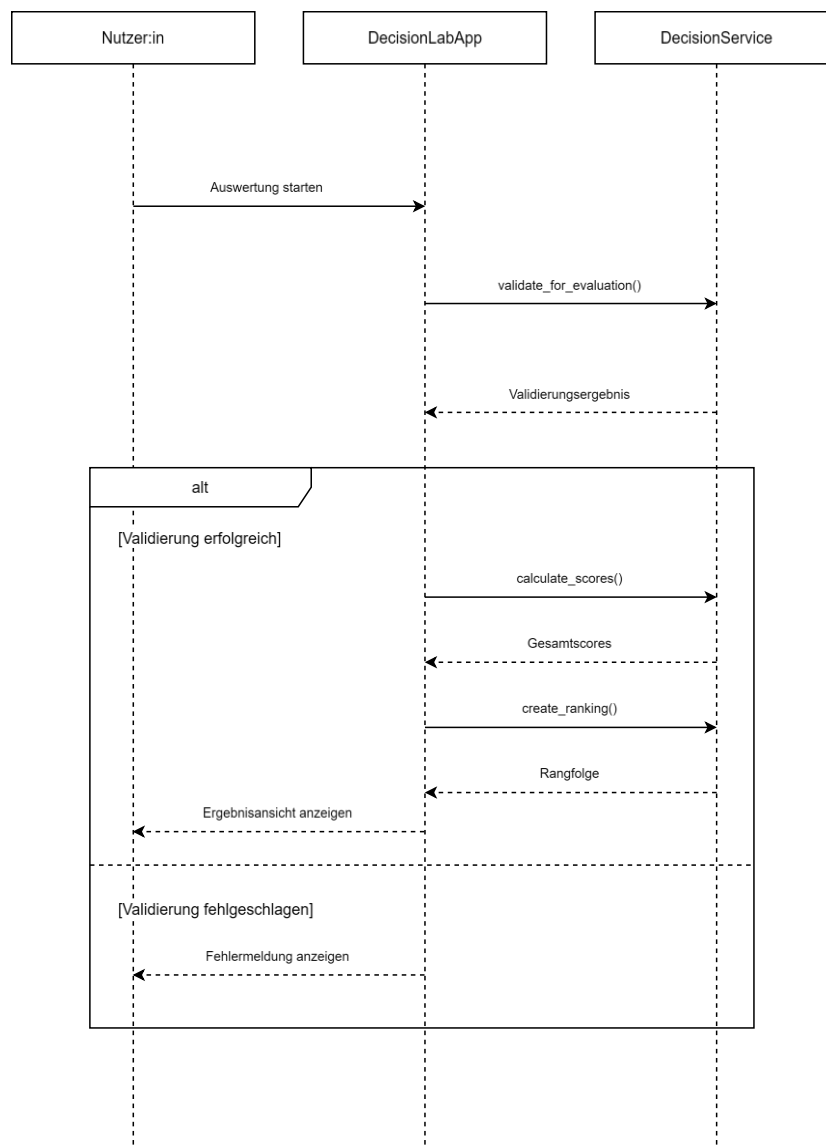
Quelle: Eigene Darstellung

4 Dynamisches Verhalten

4.1 Sequenzdiagramm der Auswertung

Der zentrale Ablauf der Auswertung wird durch ein UML-Sequenzdiagramm dargestellt. Nach dem Start der Auswertung veranlasst die Benutzeroberfläche zunächst die Prüfung der fachlichen Voraussetzungen durch den DecisionService. Sind die Voraussetzungen erfüllt, werden die gewichteten Gesamtscores berechnet und anschließend in eine Rangfolge überführt. Die Benutzeroberfläche stellt das daraus resultierende Ergebnis dar. Sind die Voraussetzungen für eine Auswertung nicht erfüllt, wird keine Berechnung durchgeführt und die Benutzeroberfläche zeigt stattdessen eine entsprechende Fehlermeldung an.

Abbildung 3: UML-Sequenzdiagramm "Auswertung durchführen"



Quelle: Eigene Darstellung